



Swagger &



PLANO DE AULA PRÁTICO

Integração e uso do Swagger em uma API Node.js

PROJETO NOVO — CRIADO DO ZERO

Node.js + Express + OpenAPI + Swagger UI

Duração: 1h10min (70 minutos)

Público: iniciantes/intermediários em desenvolvimento de APIs

Objetivo geral

Criar uma API REST simples do zero, sem reaproveitar nenhum projeto anterior, instalando as ferramentas necessárias, criando as rotas, integrando OpenAPI/Swagger UI, documentando a API e testando os endpoints diretamente no navegador.

MUDANÇA PRINCIPAL DESTA VERSÃO

O plano não parte de um projeto existente. O aluno começa com uma pasta vazia e executa todos os comandos para criar o projeto api-swagger-aula. Os blocos de código foram reformulados para usar texto preto sobre fundo branco, facilitando impressão e leitura.

2. Visão geral da aula

A divisão abaixo totaliza exatamente 70 minutos.

Tempo	Etapa	Professor faz	Aluno faz
0-5	Conceitos	Explica API, rota, endpoint, Swagger, OpenAPI e Swagger UI.	Acompanha e responde perguntas rápidas.
5-10	Ambiente	Confere Node/npm e mostra terminal + VS Code.	Verifica ferramentas.
10-18	Novo projeto	Cria pasta, npm init e instala dependências.	Repete os comandos.
18-32	API	Cria API de produtos com CRUD em memória.	Cria app.js.
32-45	Swagger	Cria swagger.js e explica OpenAPI/@openapi.	Configura documentação.
45-55	Swagger UI	Inicia servidor e demonstra documentação.	Acessa /api-docs.
55-62	Testes	Demonstra GET, POST, PUT e DELETE.	Executa pelo Try it out.
62-67	Atividade	Propõe GET /categorias e orienta.	Cria/documenta/testa a rota.
67-70	Revisão	Checklist + perguntas + mercado.	Responde e verifica resultado.

CONTROLE DO TEMPO

A atividade prática é curta de propósito. Se a turma atrasar, o professor demonstra a solução de GET /categorias e mantém a revisão final dentro dos 70 minutos.

3. Pré-requisitos

- Computador com Windows, macOS ou Linux.
- Internet para instalar Node.js e os pacotes npm.
- Node.js LTS — no momento da preparação, a linha LTS oficial é 24.21.0.
- npm — instalado junto com Node.js.
- VS Code ou outro editor de código.
- Navegador para abrir o Swagger UI.
- PowerShell/Terminal para executar os comandos.

3.1 Verificar as ferramentas

Execute no PowerShell/Terminal

```
node -v
npm -v
code --version
```

VERSÕES ATUAIS CONSULTADAS

O site oficial do Node.js informa v24.21.0 LTS e v26.8.2 Current. Para uma aula, prefira a LTS. O npm informado para Node 24.21.0 é 11.19.0. As versões podem mudar depois da publicação do material.

3.2 Se Node.js não estiver instalado

- Abra o navegador e procure Node.js Downloads no site oficial.
- Escolha a versão marcada como LTS.
- No Windows, execute o instalador e mantenha as opções padrão.
- Feche e abra novamente o PowerShell.
- Execute node -v e npm -v novamente.

ERRO COMUM

Se o terminal disser que “node” ou “npm” não é reconhecido, normalmente o Node.js não está instalado ou o terminal foi aberto antes da instalação. Feche o terminal, abra outro e teste novamente.

4. Instalação e configuração do novo projeto

4.1 Abrir o PowerShell

- Pressione Win + R.
- Digite powershell.

- Pressione Enter.

4.2 Criar uma pasta totalmente nova

```
mkdir api-swagger-aula
cd api-swagger-aula
npm init -y
```

O que aconteceu?

- mkdir criou a pasta.
- cd entrou na pasta.
- npm init -y criou o package.json inicial.

4.3 Instalar as ferramentas da aula

```
npm install express swagger-jsdoc swagger-ui-express
```

Função de cada pacote:

Pacote	Função
express	Cria o servidor e as rotas HTTP.
swagger-jsdoc	Lê comentários @openapi e gera a especificação OpenAPI.
swagger-ui-express	Publica o Swagger UI dentro do Express.

4.4 Abrir no VS Code

```
code .
```

RESULTADO ESPERADO

No VS Code você deve ver package.json, package-lock.json e node_modules. Ainda não existe app.js nem swagger.js: eles serão criados agora.

5. Criação da API — projeto novo

5.1 Criar app.js

- No VS Code, no Explorer, clique em New File.
- Digite app.js.
- Cole o código completo abaixo.

app.js — código completo

```
const express = require('express');
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./swagger');

const app = express();
const PORT = 3000;

app.use(express.json());

let produtos = [
  { id: 1, nome: 'Teclado', preco: 120 },
  { id: 2, nome: 'Mouse', preco: 80 }
];

let proximoId = 3;

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

/**
 * @openapi
 * /produtos:
 *   get:
 *     tags: [Produtos]
 *     summary: Lista todos os produtos
 *     responses:
 *       200:
 *         description: Lista de produtos
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/Produto'
 */
app.get('/produtos', (req, res) => {
```

```

    res.status(200).json(produtos);
  });

  /**
   * @openapi
   * /produtos/{id}:
   *   get:
   *     tags: [Produtos]
   *     summary: Busca um produto pelo ID
   *     parameters:
   *       - in: path
   *         name: id
   *         required: true
   *         schema:
   *           type: integer
   *     responses:
   *       200:
   *         description: Produto encontrado
   *       404:
   *         description: Produto não encontrado
   */
  app.get('/produtos/:id', (req, res) => {
    const id = Number(req.params.id);
    const produto = produtos.find((item) => item.id === id);

    if (!produto) {
      return res.status(404).json({ mensagem: 'Produto não encontrado.' });
    }

    res.status(200).json(produto);
  });

  /**
   * @openapi
   * /produtos:
   *   post:
   *     tags: [Produtos]
   *     summary: Cadastra um produto
   *     requestBody:
   *       required: true
   *       content:
   *         application/json:
   *           schema:
   *             $ref: '#/components/schemas/NovoProduto'
   *           example:
   *             nome: "Monitor"
   *             preco: 900
   *     responses:
   *       201:
   *         description: Produto criado
   *       400:
   *         description: Dados inválidos
   */
  app.post('/produtos', (req, res) => {
    const { nome, preco } = req.body;

    if (!nome || preco === undefined) {
      return res.status(400).json({
        mensagem: 'nome e preco são obrigatórios.'
      });
    }

    const novoProduto = {
      id: proximoId++,
      nome,
      preco: Number(preco)
    };

    produtos.push(novoProduto);
    res.status(201).json(novoProduto);
  });

  /**
   * @openapi
   * /produtos/{id}:
   *   put:
   *     tags: [Produtos]
   *
   *     summary: Atualiza um produto
   *     parameters:
   *       - in: path
   *         name: id
   *         required: true
   *         schema:
   *           type: integer
   *     requestBody:
   *       required: true
   *       content:

```

```

*       application/json:
*       schema:
*         $ref: '#/components/schemas/NovoProduto'
*       example:
*         nome: "Monitor Full HD"
*         preco: 950
*     responses:
*       200:
*         description: Produto atualizado
*       404:
*         description: Produto não encontrado
*/
app.put('/produtos/:id', (req, res) => {
  const id = Number(req.params.id);
  const indice = produtos.findIndex((item) => item.id === id);

  if (indice === -1) {
    return res.status(404).json({ mensagem: 'Produto não encontrado.' });
  }

  const { nome, preco } = req.body;

  produtos[indice] = {
    id,
    nome,
    preco: Number(preco)
  };

  res.status(200).json(produtos[indice]);
});

/**
 * @openapi
 * /produtos/{id}:
 *   delete:
 *     tags: [Produtos]
 *     summary: Exclui um produto
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema:
 *           type: integer
 *     responses:
 *       204:
 *         description: Produto excluído
 *       404:
 *         description: Produto não encontrado
 */
app.delete('/produtos/:id', (req, res) => {
  const id = Number(req.params.id);
  const indice = produtos.findIndex((item) => item.id === id);

  if (indice === -1) {
    return res.status(404).json({ mensagem: 'Produto não encontrado.' });
  }

  produtos.splice(indice, 1);
  res.status(204).send();
});

app.listen(PORT, () => {
  console.log(`API rodando em http://localhost:${PORT}`);
  console.log(`Swagger UI em http://localhost:${PORT}/api-docs`);
});

```

5.2 O que a API faz

Método	Endpoint	Função	Entrada
GET	/produtos	Lista produtos	Nenhuma
GET	/produtos/:id	Busca produto	id na URL
POST	/produtos	Cria produto	JSON no body
PUT	/produtos/:id	Atualiza produto	id + JSON
DELETE	/produtos/:id	Exclui produto	id na URL

5.3 Conceitos usados

- Rota: regra que associa um método HTTP e um caminho a uma função.
- Endpoint: ponto de acesso, como GET /produtos.
- Parâmetro de rota: valor variável na URL, como :id.

- Body: dados enviados no corpo da requisição; aqui usamos JSON.
- Status HTTP: informa o resultado, como 200, 201, 204, 400 e 404.
- Memória: o array produtos funciona como um banco temporário; ao reiniciar o servidor, os dados voltam ao estado inicial.

6. Integração do Swagger/OpenAPI

6.1 OpenAPI x Swagger

- OpenAPI é a especificação usada para descrever formalmente uma API.
- Swagger é o ecossistema de ferramentas que trabalha com essa descrição.
- Swagger UI transforma a descrição OpenAPI em uma documentação visual e interativa.
- swagger-jsdoc gera a especificação a partir dos comentários do código.
- swagger-ui-express disponibiliza a interface dentro do Express.

6.2 Criar swagger.js

- No VS Code, crie um arquivo chamado swagger.js.
- Cole o código abaixo.

swagger.js — código completo

```
const swaggerJSDoc = require('swagger-jsdoc');

const options = {
  definition: {
    openapi: '3.0.3',
    info: {
      title: 'API de Produtos - Aula de Swagger',
      version: '1.0.0',
      description: 'API simples criada do zero para demonstrar OpenAPI e Swagger UI.'
    },
    servers: [
      {
        url: 'http://localhost:3000'
      }
    ],
    tags: [
      {
        name: 'Produtos',
        description: 'Operações de produtos'
      }
    ],
    components: {
      schemas: {
        Produto: {
          type: 'object',
          required: ['id', 'nome', 'preco'],
          properties: {
            id: {
              type: 'integer',
              example: 1
            },
            nome: {
              type: 'string',
              example: 'Teclado'
            },
            preco: {
              type: 'number',
              example: 120
            }
          }
        },
        NovoProduto: {
          type: 'object',
          required: ['nome', 'preco'],
          properties: {
            nome: {
              type: 'string',
              example: 'Monitor'
            },
            preco: {
              type: 'number',
              example: 900
            }
          }
        }
      }
    }
  }
};

const swaggerSpec = swaggerJSDoc(options);
```

```
apis: ['./app.js'],
failOnErrors: true
};

module.exports = swaggerJSDoc(options);
```

6.3 Por que o app.js tem comentários @openapi?

O Swagger não “adivinha” tudo sobre uma rota. O comentário @openapi informa método, endpoint, parâmetros, body, respostas e schemas. O swagger-jsdoc lê esses comentários e monta a especificação.

PONTO CRÍTICO

A indentação dentro dos blocos @openapi é importante porque a estrutura segue YAML. Se houver erro de indentação, o swagger-jsdoc pode falhar.

7. Iniciar e abrir a documentação

7.1 Iniciar o servidor

```
node app.js
```

RESULTADO ESPERADO

O terminal deve informar API rodando em <http://localhost:3000> e Swagger UI em <http://localhost:3000/api-docs>.

7.2 Abrir o Swagger UI

- Abra o navegador.
- Digite <http://localhost:3000/api-docs>.
- Pressione Enter.

RESULTADO ESPERADO

A página do Swagger UI deve aparecer com o título API de Produtos - Aula de Swagger e as operações GET, POST, PUT e DELETE.

7.3 Fluxo completo da ferramenta

COMO EXPLICAR NO QUADRO

Node.js executa JavaScript → Express cria servidor/rotas → OpenAPI descreve a API → swagger-jsdoc gera a especificação → Swagger UI mostra a documentação → Try it out permite testar.

8. Testes práticos pelo Swagger UI

O aluno deve executar as ações abaixo no navegador. O terminal continua aberto executando `node app.js`.

8.1 GET /produtos

- Expanda GET /produtos.
- Clique em Try it out.
- Clique em Execute.
- Leia o Server response.

ESPERADO

Status 200 e um array contendo Teclado e Mouse.

8.2 GET /produtos/{id}

- Expanda a operação.
- Clique em Try it out.
- Digite 1 no campo id.
- Clique em Execute.

ESPERADO

Status 200 e o produto Teclado. Repita com 999 para demonstrar o 404.

8.3 POST /produtos

- Clique em Try it out.
- No Request body, use o JSON abaixo.

```
{
  "nome": "Monitor",
  "preco": 900
}
```

- Clique em Execute.
- Observe o status e o ID criado.

ESPERADO

Status 201 Created e um novo produto, normalmente com id 3.

8.4 PUT /produtos/{id}

- Clique em Try it out.
- Digite 1 no id.
- Use o body abaixo.

```
{
  "nome": "Teclado Mecânico",
  "preco": 250
}
```

Clique em Execute.

ESPERADO

Status 200 OK e o produto 1 atualizado.

8.5 DELETE /produtos/{id}

- Clique em Try it out.
- Digite 2.
- Clique em Execute.

ESPERADO

Status 204 No Content. Não haverá JSON no corpo da resposta.

9. Documentação das rotas

Rota	Parâmetros	Body	Sucesso	Erro
GET /produtos	—	—	200 + array	—
GET /produtos/{id}	id: integer	—	200 + objeto	404
POST /produtos	—	nome, preco	201 + objeto	400
PUT /produtos/{id}	id: integer	nome, preco	200 + objeto	404
DELETE /produtos/{id}	id: integer	—	204	404

9.1 Exemplo de resposta

```
{
  "id": 3,
  "nome": "Monitor",
  "preco": 900
}
```

9.2 Status que o aluno deve reconhecer

Código	Significado	Exemplo na aula
200	OK	GET e PUT concluídos.
201	Created	POST criou produto.
204	No Content	DELETE excluiu produto.
400	Bad Request	POST sem nome/preço.
404	Not Found	ID inexistente.

10. Atividade prática — nova rota

Objetivo: criar uma rota diferente das cinco demonstradas, documentá-la e testá-la sem ajuda direta do professor.

Tarefa: GET /categorias

- Criar uma rota GET /categorias.
- Retornar um array com pelo menos três categorias.
- Adicionar o bloco @openapi.
- Salvar.
- Reiniciar o servidor com node app.js.
- Abrir /api-docs e testar com Try it out → Execute.

Solução esperada para o professor

```
/**
 * @openapi
 * /categorias:
 *   get:
 *     tags: [Categorias]
 *     summary: Lista as categorias
 *     responses:
 *       200:
 *         description: Lista de categorias
 */
app.get('/categorias', (req, res) => {
  const categorias = [
    { id: 1, nome: 'Periféricos' },
    { id: 2, nome: 'Monitores' },
    { id: 3, nome: 'Acessórios' }
  ];

  res.status(200).json(categorias);
});
```

RESULTADO

A operação GET /categorias deve aparecer no Swagger UI e retornar 200 com três objetos JSON.

11. Troubleshooting

Problema	CAUSA	COMO IDENTIFICAR	COMO CORRIGIR
node/npm não reconhecido	Node.js não instalado ou PATH não atualizado.	node -v / npm -v falham.	Instale o LTS e abra um novo terminal.
npm install falhou	Internet, permissão ou problema de cache.	Erro aparece durante instalação.	Corrija a conexão/permissão e execute npm install novamente.
Cannot find module	Dependência ausente ou projeto aberto em pasta errada.	Erro ao rodar node app.js.	Confirme a pasta com Get-Location e execute npm install.
Swagger não abre	Servidor parado ou URL errada.	localhost:3000/api-docs não carrega.	Execute node app.js e use exatamente /api-docs.
Rota não aparece	Comentário @openapi inválido ou apis aponta para arquivo errado.	Swagger UI não lista a rota.	Confira @openapi, indentação e apis: ['./app.js'].
Endpoint aparece, mas falha	Rota documentada não corresponde ao código.	Try it out retorna erro.	Compare método/caminho do Swagger com app.get/post/put/delete.
JSON inválido	Aspas/chaves/vírgulas incorretas.	Request body inválido.	Use JSON com aspas duplas e sem vírgula final.
Porta 3000 ocupada	Outro programa já usa a porta.	Erro EADDRINUSE.	Feche o processo ou altere PORT para 3001 e atualize servers.url.

Diagnóstico rápido de porta no Windows

```
netstat -ano | findstr :3000
```

Se for seguro encerrar o processo, use o PID mostrado:

```
taskkill /PID NUMERO_DO_PID /F
```

12. Conexão com o mercado

- Frontend + backend: a documentação mostra exatamente como consumir a API.
- Equipes: reduz dúvidas sobre endpoints, parâmetros, body e respostas.
- Manutenção: novos integrantes entendem a API mais rapidamente.
- Robustez: respostas e códigos HTTP documentados tornam o comportamento previsível.
- Escalabilidade: uma documentação padronizada ajuda quando a API cresce e várias pessoas trabalham nela.

Exemplos profissionais

APIs de sistemas internos, aplicativos móveis, e-commerces, microserviços e integrações entre empresas podem usar OpenAPI para estabelecer um contrato comum entre consumidores e servidores.

13. Revisão final

O que é uma API?

Resposta esperada: Uma interface para comunicação entre sistemas.

O que é Swagger?

Resposta esperada: Conjunto/ecossistema de ferramentas associado à documentação de APIs.

O que é OpenAPI?

Resposta esperada: Especificação para descrever uma API.

Para que serve Swagger UI?

Resposta esperada: Para visualizar e testar uma API documentada.

O que é endpoint?

Resposta esperada: Um ponto de acesso da API, formado por método e caminho.

Como testar uma rota?

Resposta esperada: Try it out → preencher → Execute → interpretar resposta.

Por que documentar?

Resposta esperada: Para facilitar integração, manutenção, entendimento e evolução.

14. Checklist final

Item	OK
☐ Node.js instalado	☐
☐ npm funcionando	☐
☐ Pasta nova criada	☐
☐ npm init executado	☐
☐ Express instalado	☐
☐ swagger-jsdoc instalado	☐
☐ swagger-ui-express instalado	☐
☐ app.js criado	☐
☐ swagger.js criado	☐
☐ API iniciada	☐
☐ Swagger UI aberto	☐
☐ GET testado	☐
☐ POST testado	☐
☐ PUT testado	☐
☐ DELETE testado	☐
☐ Nova rota criada/documentada	☐

15. Roteiro minuto a minuto do professor

Tempo	Papel	Ação	Papel	Ação
0-2	PROFESSOR FAZ	Apresenta objetivo e imagem: documentação facilita uso da API.	ALUNO FAZ	Acompanha.

Tempo	Papel	Ação	Papel	Ação
2-5	PROFESSOR FAZ	Define API, rota, endpoint, Swagger, OpenAPI e UI.	ALUNO FAZ	Responde exemplos.
5-7	PROFESSOR FAZ	Confere node -v/npm -v.	ALUNO FAZ	Executa comandos.
7-10	PROFESSOR FAZ	Mostra terminal, VS Code e estrutura vazia.	ALUNO FAZ	Abre ambiente.
10-13	PROFESSOR FAZ	Cria pasta + cd + npm init.	ALUNO FAZ	Repete.
13-18	PROFESSOR FAZ	Instala três dependências e abre VS Code.	ALUNO FAZ	Instala.
18-22	PROFESSOR FAZ	Cria app.js e explica Express.	ALUNO FAZ	Cria/cola.
22-27	PROFESSOR FAZ	Explica GET e GET por ID.	ALUNO FAZ	Confere.
27-32	PROFESSOR FAZ	Explica POST/PUT/DELETE e status.	ALUNO FAZ	Finaliza API.
32-36	PROFESSOR FAZ	Explica OpenAPI x Swagger.	ALUNO FAZ	Acompanha.
36-41	PROFESSOR FAZ	Cria swagger.js e explica schemas.	ALUNO FAZ	Cria/cola.
41-45	PROFESSOR FAZ	Inicia servidor e abre /api-docs.	ALUNO FAZ	Confere UI.
45-48	PROFESSOR FAZ	Demonstra GET.	ALUNO FAZ	Testa.
48-51	PROFESSOR FAZ	Demonstra GET por ID.	ALUNO FAZ	Testa 1/999.
51-54	PROFESSOR FAZ	Demonstra POST.	ALUNO FAZ	Envia JSON.
54-57	PROFESSOR FAZ	Demonstra PUT.	ALUNO FAZ	Atualiza.
57-60	PROFESSOR FAZ	Demonstra DELETE.	ALUNO FAZ	Exclui.
60-62	PROFESSOR FAZ	Explica status e leitura da resposta.	ALUNO FAZ	Interpreta.
62-65	PROFESSOR FAZ	Passa GET /categorias.	ALUNO FAZ	Cria/documenta.
65-67	PROFESSOR FAZ	Corrige atividade rapidamente.	ALUNO FAZ	Testa.
67-70	PROFESSOR FAZ	Revisão + checklist + mercado.	ALUNO FAZ	Responde.

16. Comandos e arquivos — folha de consulta

Comandos completos para começar um projeto novo

```
mkdir api-swagger-aula
cd api-swagger-aula
npm init -y
npm install express swagger-jsdoc swagger-ui-express
code .
node app.js
```

Estrutura final

```
api-swagger-aula/
├── node_modules/
├── app.js
├── package-lock.json
├── package.json
└── swagger.js
```

URLs

- Servidor: <http://localhost:3000>
- Swagger UI: <http://localhost:3000/api-docs>

- API: <http://localhost:3000/produtos>

17. Versões e referências técnicas

Versões verificadas em 15/09/2026:

Ferramenta	Versão/estado	Observação
Node.js	24.21.0 LTS	Linha recomendada para estabilidade.
Node.js	26.8.2 Current	Linha Current; não é a escolhida para a aula.
Express	5.2.1	Framework usado no projeto.
swagger-jsdoc	6.3.0	Tag latest consultada no npm.
swagger-ui-express	5.0.1	Integra Swagger UI ao Express.
OpenAPI	3.0.3 no projeto	Formato escolhido para manter o exemplo didático e compatível.

Fontes oficiais/primárias

- Node.js Downloads — nodejs.org
- Express — pacote oficial publicado no npm
- swagger-jsdoc — npm
- swagger-ui-express — npm
- OpenAPI Specification — spec.openapis.org

Observação: versões de pacotes npm podem mudar. Em uma nova edição da aula, confirme as versões novamente e mantenha o package-lock.json para reproduzir a instalação.